Cours Logique Formelle

Nourhene Hanana

كلية العلوم بتونس
FACULTE DES SCIENCES DE TUNIS

PLAN

1. Chapitre 1: Introduction à la logique propositionnelle
2. Chapitre 2: Calcul Propositionnel d'ordre 0 (CP0)
3. Chapitre 3: Calcul Propositionnel d'ordre 1 (Calcul des prédicats)
4. Chapitre 4: Méthodes de Résolution et Unification

Chapitre 1: Introduction à la logique propositionelle

La logique, c’est quoi ?

- Le mot logique provient du mot grec logos (raison, discours) et signifie « science de la raison »

- Il s’agit d’une discipline qui étudie les règles du raisonnement valide et permet de distinguer les arguments corrects des arguments incorrects, en se basant sur la structure des propositions, les relations entre elles et non pas sur leur contenu.

But: l’art de bien raisonner pour distinguer le vrai du faux raisonnement.

Logique & Histoire

- Antiquité grecque: (Logique est un outil philosophique)

Aristote considéré comme le père de la logique fait naitre au raisonnement déductif de la forme:

Tous les hommes sont mortels , Socrate est un homme ▫ Donc Socrate est mortel

- Moyen âge: (Logique scolastique)

Les médiévaux analysent finement le sens des termes dans une proposition. Ils développent la théorie des suppositions ou le fait qu'un mot se tienne pour un individu, une classe ou une idée.

Exemple: dans "l'homme est intelligent", le terme "homme" suppose pour toute l'éspèce humaine. Par contre dans "cet homme est un philosophe", "homme" suppose pour un individu particulier ▫ analyse sémantique

Logique & Histoire

- XIXe siècle: Logique est une science mathématique

George Boole introduit l'algèbre booléenne qui exprime le raisonnement en termes d'opérations algébriques (ET, OU, NON). Ainsi que Frege introduit la logique des prédicats dépassant largement la logique aristotélicinne.

- 20ème siècle: Logique un pilier de l'informatique moderne (Turing, IA)

- Aujourd'hui:

Logique formelle est partout:

- Mathématiques: prevue, théorie des ensembles, modèles
- Informatique: algorithmes, langage de programmation, verification de programmes, intelligence artificielle
- Philosophie: analyse des arguments ..

Langage & Logique

- Le langage est un système de signes: mots appartenant à un alphabet, construit d'un ensemble de symboles ainsi que l'ensemble de règles de construction. On distingue:

- Langage naturel: nos langues (francais, arabe, anglais ..) riches mais ambigues.
- Langage formel: un système de symboles artificiels (mathématiques, langage logique, langage informatique) conçu pour être précis et sans ambiguité.

- Relation entre logique et langage:
- La logique: étude de la validité du raisonnement
- Le langage: moyen de formuler et communiquer des pensées.

Lien:

- Sans langage, la logique ne peut pas être exprimée.
- Sans logique, le langage resterait ambigu et imprécis.

Chapitre 2: Calcul Propositionnel d'ordre 0

PLAN Chapitre 2: Calcul Propositionnel d'ordre 0

I. **Syntaxe**
   - Définition du langage
   - Arbre de décomposition d'une formule
   - Substitution dans une formule

II. Sémantique

Syntaxe : Introduction

- **But**: étudier les relations entre énoncés, propositions et formules.
- **Un énoncé**: phrase en langage naturel.

Exemple: « Il pleut »

- **Une proposition**: sens de l'énoncé, ce qu'il affirme avec une valeur de vérité (vrai/faux)

Exemple: « Il pleut » est vrai ou faux selon la météo

- **Une formule**: représentation symbolique d'une proposition dans un langage logique.

Exemple: p: « Il pleut » q: « J'ai un parapluie »

La formule p ➔ q = « S'il pleut, alors j'ai un parapluie »

- **Pouquoi on l'appelle CP0 ?**

On manipule uniquement des propositions entières (phrases qui sont soit vraies, soit fausses).

Exemple :

- p: « Il pleut »
- q: « Je prends mon parapluie »
- Formule : $p \rightarrow q$(*si il pleut, alors je prends mon parapluie*).

➔ On ne regarde pas l'intérieur des phrases (« il pleut » est une boîte noire).

On combine seulement des propositions avec des connecteurs : $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$.

Syntaxe : Introduction

- Les formules sont de 2 types :

- Atomique (Exemple : “il pleut”)
- Composée d’autres forumles à l’aide de connecteurs logiques propositionnels

- Les connecteurs logiques sont :

¬ : NON

∨ : OU

∧ : ET

⟹ : Implique

⇔ : Equivalent

Syntaxe: Définitions

- Une formule atomique: une formulee qui ne contient aucun connecteur logique. Elle prend les valeurs vrai ou faux et donne une information sur un état de chose.

- Une formule composée peut être obtenue à partir de:

- Des formules atomique

Exemple : « Il pleut, le soleil se cache » est une formule composée de deux formules atomiques: “il pleut” et “le soleil se cache” et se traduit par : il pleut $\Longrightarrow$ le soleil se cache

- D’autres formules composées

Exemple: « Il pleut, le soleil se cache alors je ne sors pas » est une formule composée d’autres formules composée il pleut $\Longrightarrow$ le soleil se cache$\Longrightarrow$ je ne sors pas.

- Note: la valeur de vérité d’une formule composée se calcule comme une fonction des formules qui la composent.

Syntaxe: CP0

Le langage du calcul propositionnel (CP0) est formé de :

- un ensemble fini dénombrable de propositions P = {*p*1,*p*2,...}
- un ensemble de connecteurs logiques {¬, ∨, ∧, ⇒ ⇔}
- Parentheses
- Constantes : 1(*Vrai*,*V*,⊤) et 0(*Faux*,*F*,⊥)

L'ensemble $\mathcal{F}$ des formules du calcul propositionnel est le plus petit ensemble tel que :

- tout symbole propositionnel est une formule ;
- si φ est une formule alors $\neg\varphi$ est une formule ;
- si φ et ψ sont des formules alors $\varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \Rightarrow \psi$ et $\varphi \Leftrightarrow \psi$ sont des formules.

Syntaxe: CP0

- Les parenthèses sont nécessaires pour lever les ambiguïtés lorsque plusieurs connecteurs de même priorité apparaissent dans une formule.

Exemple: Ayant cet énoncé: Il pleut et il neige ou il fait chaud.

p: Il pleut q: Il neige r: il fait chaud

Sans parentheses: p ∧ q ∨ r

Formule 1 : (p ∧ q) ∨ r ▫ Si r est vraie alors toute la proposition est vraie

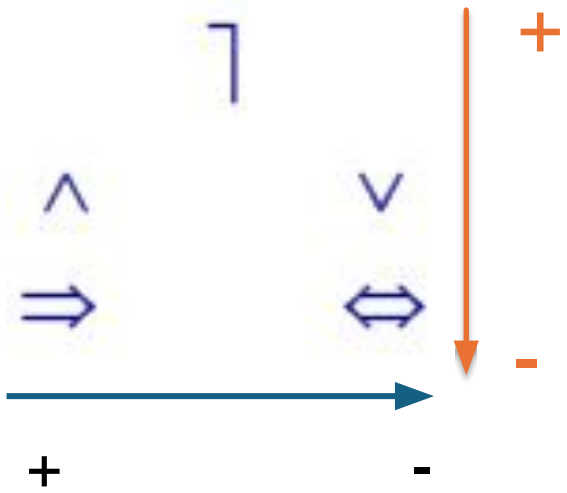
Formule 2 : p ∧ (q ∨ r) ▫ Si p est fausse alors toute la proposition est fausse.

Toute fois un énoncé doit être modélisé d'une seule façon et doit avoir une seule valeur (vrai ou faux) ▫ Les parenthèses sont nécessaires.

Syntaxe: CP0

- Il faut fixer une priorité (poids) pour les opérateurs.

Ordre de priorité:



Par Convention
Priorité la plus faible

Par coutume:

ensemble de conventions admises par les logiciens pour éviter de mettre trop de parenthèses (càd que ca peut changer, il s'agit juste d'un accord)

Syntaxe: CP0

Exemple 1: p ∧ q ⇒ r ∧ s ⇒ ⌉ p ⇔ u ∨ v

((((p ∧ q) ⇒ (r ∧ s)) ⇒ (⌉ p)) ⇔ (u ∨ v))

2 5 3 6 1 7 4

Exemple 2: p ⇒ (q ⇔ r) ∨ s ∨ t ∧ p ⇒ ⌉ (p ∨ r) ⇔ t

(((p ⇒ (((q ⇔ r) ∨ s) ∨ (t ∧ p))) ⇒ (⌉ (p ∨ r))) ⇔ t)

Syntaxe: CP0

- On a des cas où on peut enlever les parenthèses aisément lorsqu'on peut lever l'ambiguité à l'aide de l'ordre de priorité tel que¬ p ∨ q ou bien lorsqu'on a des formules de sorte :
  - p ∧ q ∧ r parceque ∧ est associatif (Il joue aux cartes, il mange des glaces et il gagne aux échecs) ▫ la proposition est toujours vraie lorsque toutes les formules atomiques sont vraies quelque soit leurs places.

Syntaxe CP0: Arbre de décomposition

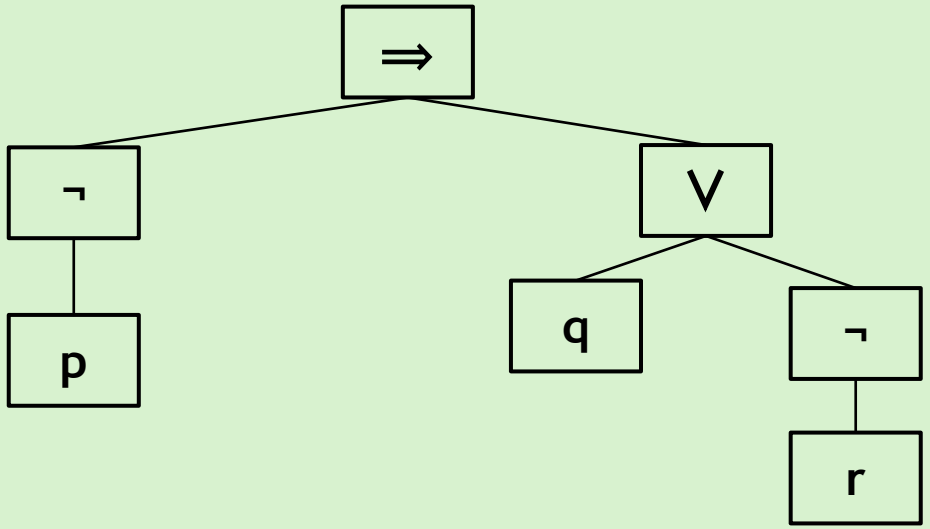
Une formule peut être présentée comme un arbre:

- Feuilles: symbôles propositionnelles
- Nœuds: les connecteurs

L'**arbre de décomposition** est utilisé en logique formelle pour représenter la structure hiérarchique d'une formule. Cet arbre sert principalement pour clarifier la construction d'une formule, éviter les ambiguïtés et faciliter le raisonnement logique.

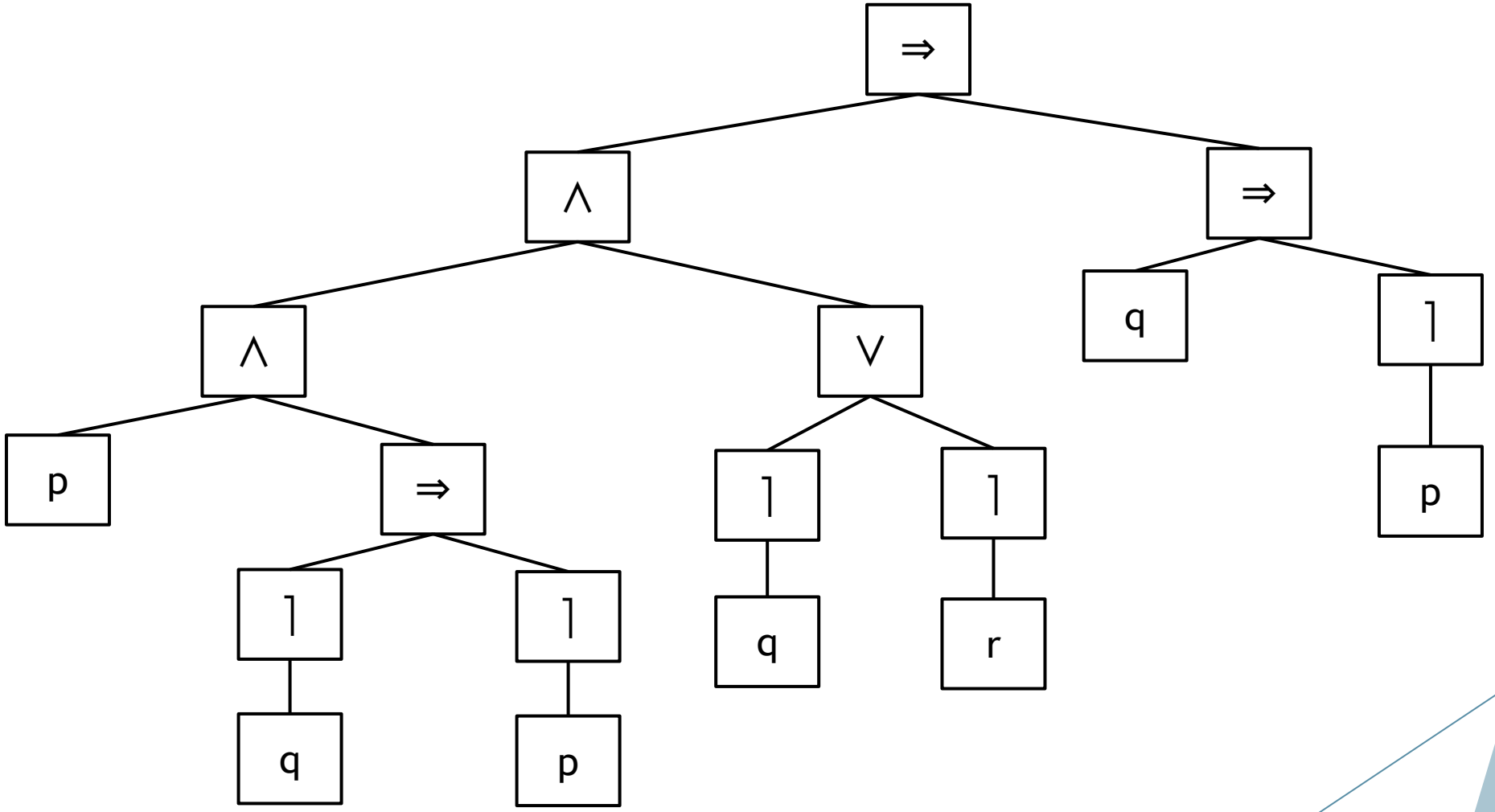
Exemple:

La formule $\neg p \Rightarrow (q \vee \neg r)$ correspond à l'arbre suivant:



Syntaxe CP0: Arbre de décomposition

Exemple 2 ((p ∧ (⌉ q ⇒ ⌉ p)) ∧ (⌉ q ∨ ⌉ r)) ⇒ (q ⇒ ⌉ p)



Syntaxe CP0: Arbre de décomposition

- Corollaire:
- L'arbre de décomposition d'une formule est unique
- On dit que le langage des propositions est non ambigu

- Notion de sous formules:

Une sous-formule est tout fragment d'une formule qui, pris isolément, reste une formule bien formée.

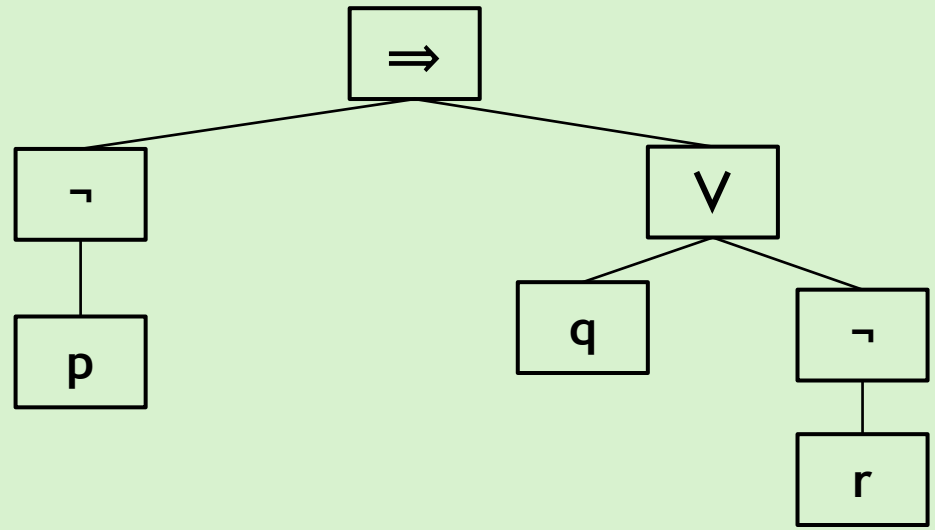
- $\mathcal{SF}(p) = \{p\}$;
- $\mathcal{SF}(\neg\varphi) = \{\neg\varphi\} \cup \mathcal{SF}(\varphi)$;
- $\mathcal{SF}(\varphi \circ \psi) = \{\varphi \circ \psi\} \cup \mathcal{SF}(\varphi) \cup \mathcal{SF}(\psi)$;(oú $\circ$ désigne $\vee, \wedge, \Rightarrow$ ou $\Leftrightarrow$).

Syntaxe CP0: Arbre de décomposition

- Notion de sous formules:

Une sous-formule est tout fragment d'une formule qui, pris isolément, reste une formule bien formée.

$SF(\neg p \Rightarrow (q \vee \neg r)) = \{p, q, r, \neg p, \neg r, q \vee \neg r, \neg p \Rightarrow (q \vee \neg r)\}$



Syntaxe du CP0

Exercice 1 formulation d'un problème:

1. Si Ali ne vient pas à l'école, alors il est malade.
2. Ali vient à l'école, il dort en classe et n'écoute pas alors il n'aura pas une bonne note
3. Leila et Ali viennent à l'école, ils auront une bonne note s'ils ne dorment pas en classe

Solution:

- p: Ali vient à l'école
- q: Ali est malade
- r: Ali dort en classe
- s: Ali écoute en classe
- t: Ali a une bonne note
- u: Leila vient à l'école
- v: Leila dort en classe
- w: Leila a une bonne note

- Phrase 1: $\neg p \Longrightarrow q$
- Phrase 2: $p \wedge r \wedge \neg s \Longrightarrow \neg t$
- Phrase 3: $((p \wedge u) \wedge (\neg v \wedge \neg r)) \Longrightarrow (t \wedge w)$

Syntaxe : CP0

- **Exercice 2: Forumlation de problème:**

Enoncé: « Si Ali révise ou Leila révise et que le professeur est strict ou le cours est long, alors les étudiants doivent étudier sérieusement. »

Questions:

1. Identifier les propositions de base atomiques et les nommer
2. Traduire l'énoncé en formule logique CP0 correctement parenthésée
3. Montrer que les parenthèses sont nécessaires en proposant une interprétation erronée si on les omet
4. Construire les sous formules de la formule complète
5. Construire l'arbre correspondante à la formule complète

Syntaxe : CP0

- **Exercice 2: Forumlation de problème:**

1. p: ali révise q: leila révise r: professeur est strict s: cours est long t: les étudiants doivent étudier sérieusement
2. A: Ali révise ou Leila révise: $p \vee q$

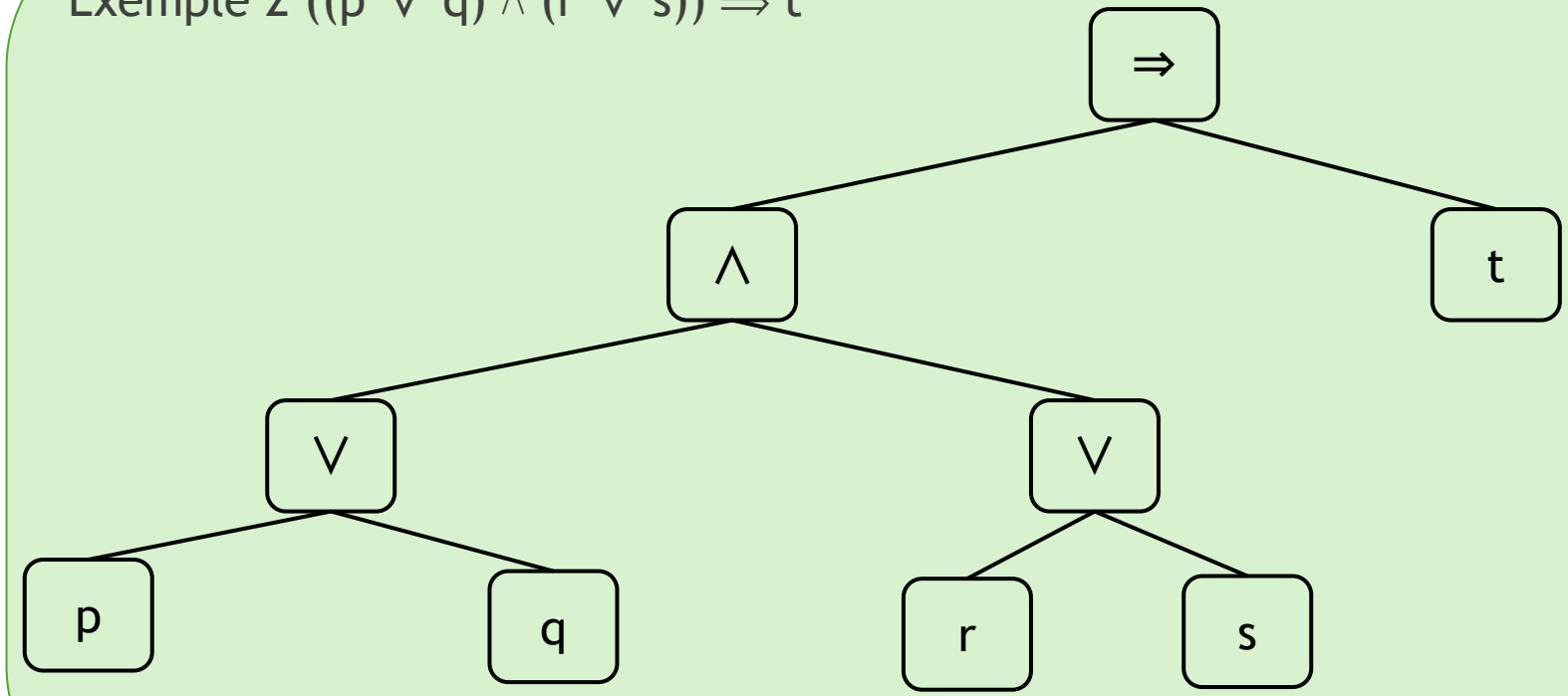
B: Professeur est strict ou le cours est long: $r \vee s$

Si $A \wedge B$ alors les étudiants doivent étudier sérieusement: $((p \vee q) \wedge (r \vee s)) \Rightarrow t$

3. $(p \vee (q \wedge r) \vee s) \Rightarrow t$: Il suffit que s ou bien p est vraie alors ceci fait que la 1 ère partie est vraie
4. Sous formules: $\{p, q, r, s, t, p \vee q, r \vee s, ((p \vee q) \wedge (r \vee s)) , ((p \vee q) \wedge (r \vee s)) \Rightarrow t$

Syntaxe du CP0

Exemple 2 $((p \vee q) \wedge (r \vee s)) \Rightarrow t$



Syntaxe du CP0: Substitution d'une formule

- Définition:

Soient

- A et B deux formules propositionnelles
- p une variable propositionnelle de A

A [p ← B] est le mot obtenu en substituant la formule B

à la variable p

La substitution s'applique à toutes les occurrences de la variable p

Exemple

A : $p \Rightarrow (q \vee p)$ **B** : $q \Rightarrow r$

- La variable p a 2 occurrences dans A
- La variable q a une seule occurrence dans A

$A [p \leftarrow B] = B \Rightarrow (q \vee B) = (q \Rightarrow r) \Rightarrow (q \vee (q \Rightarrow r))$

Syntaxe du CP0: Substitution d'une formule

- On peut étendre la substitution à un ensemble de formules **A [p1 ← B1 , p2 ← B2 , … , pn ← Bn]** est le mot obtenu en substituant respectivement les formules B1, B2 , …, Bn à toutes les occurrences des variables p1, p2, …, pn

- **Exemples**

A : p ∨ q **B** : q ∧ r **C** : q ∧ r ∧ p

- A [p ← B, q ← C] = B ∨ C = (q ∧ r) ∨ (q ∧ r ∧ p)
- A [r ← C] = A

Syntaxe du CP0: Substitution d'une formule

La substitution simultanée est différente de la substitution séquentielle

$$A [p1 \leftarrow B1, p2 \leftarrow B2] \neq (A [p1 \leftarrow B1]) [p2 \leftarrow B2]$$

substitution simultanée *substitution séquentielle*

A: $p \wedge q$ **B** : $p \vee q$ C : $p \Rightarrow q$

- $A [p \leftarrow B, q \leftarrow C] = (p \vee q) \wedge (p \Rightarrow q)$
- $A [q \leftarrow C, p \leftarrow B] = (p \vee q) \wedge (p \Rightarrow q)$
- $(A [p \leftarrow B]) [q \leftarrow C] = ((p \vee q) \wedge q) [q \leftarrow C] = = (p \vee (p \Rightarrow q)) \wedge (p \Rightarrow q)$
- $(A [q \leftarrow C]) [p \leftarrow B]= (p \vee (p \Rightarrow q)) [p \leftarrow B] = (p \vee q) \wedge ((p \vee q) \Rightarrow q)$

• Pour la substitution simultanée l'ordre n'est pas important
$A [p \leftarrow B , q \leftarrow C] = A [q \leftarrow C , p \leftarrow B]$
• Pour la substitution séquentielle l'ordre est important
$(A [p \leftarrow B]) [q \leftarrow C] \neq (A [q \leftarrow C]) [p \leftarrow B]$

PLAN Chapitre 2: Calcul Propositionnel d'ordre 0

I. Syntaxe

II. **Sémantique**

- 1. Définitions (Interprétation)
- 2. Satisfiabilité - Validité
- 3. Equivalence et conséquence sémantiques
- 4. Satisfiabilité d'un ensemble de formules
- 5. Application
- 6. Formes normales

Sémantique du CP0: Définitions

- En logique propositionnelle, on s'intéresse au **sens** des formules, c'est-à-dire à la signification de la description textuelle.

- Le **sens** d'une formule s'appelle son **interprétation**, qui correspond à l'**évaluation logique** de cette formule.

- L'évaluation d'une formule dépend de : l'évaluation des **formules atomiques** qui la composent et les **connecteurs logiques** utilisés ($\neg, \wedge, \vee, \rightarrow, \leftrightarrow$).

- À chaque formule atomique, on associe une **valeur de vérité** : Vrai (V) ou Faux (F), parfois codée par 1 ou 0.

- Ces valeurs sont souvent présentées sous forme de **tableau de vérité**, qui montre l'évaluation de toutes les sous-formules d'une formule composée.

Sémantique du CP0: Interprétation et évaluation

- Pour formaliser et manipuler ces évaluations, on utilise **l'algèbre de Boole**, qui définit les règles des connecteurs logiques.
- L'algèbre de Boole est formé par un ensemble de valeurs de vérité $B = \{V_B, F_B\}$ et un ensemble d'opérateurs booléens $\{ \vee_B, \wedge_B,]_B, \Rightarrow_B, \Leftrightarrow_B \}$

Attention: A ne pas confondre les opérateurs booléens aux connecteurs logique; le connecteur logique sert à construire la formule, tandis que l'opérateur booléen sert à évaluer la formule.

- $\vartheta(\neg\varphi) = 1 - \vartheta(\varphi)$
- $\vartheta(\varphi \vee \psi) = max(\vartheta(\varphi), \vartheta(\psi))$
- $\vartheta(\varphi \wedge \psi) = min(\vartheta(\varphi), \vartheta(\psi))$
- $\vartheta(\varphi \Rightarrow \psi) = 0$ *ssi* $\vartheta(\varphi) = 1$ *et* $\vartheta(\psi) = 0$
- $\vartheta(\varphi \Leftrightarrow \psi) = 1$ *ssi* $\vartheta(\varphi) = \vartheta(\psi)$

Sémantique du CP0: Interprétation et évaluation

- Table de verite des differents connecteurs

p	q	$p \wedge q$	$p \vee q$	$p \Rightarrow q$	$p \Leftrightarrow q$	$\neg p$
0	0	0	0	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	0	0
1	1	1	1	1	1	0

- Modèle d'une formule

Définition

Un modèle de φ est une valuation ϑ telle que $\vartheta(\varphi) = 1$. On note $\mathrm{mod}(\varphi)$ l'ensemble des modèles de φ.

Sémantique du CP0: Interprétation et évaluation

- Exemples :

A: (((p ⇒ q) ∧ p) ⇒ p). Mod(A)=

p	q	A
0	0	1
0	1	1
1	0	1
1	1	1

B: ((p ⇒ q) ∧ (p ∧ ⌉ q)). Mod(B) ={}

C: (p ∨ q) ∧ (p ∨ ⌉ r). Mod(C)=

p	q	r	A
0	1	0	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Sémantique du CP0: Satisfaisabilitié, Tautologie et Contradiction

Définitions:

- **Satisfaisabilité**: une formule φ est dite satisfaisable si elle admet un modèle (au moins vraie pour une valuation). $mod(\varphi) \neq \emptyset$
- **Contradiction**: une formule φ est diste insatisfaisable ou inconsistante ou contradiction si pour toute valuation φ est fausse. $mod(\varphi) = \emptyset$

Note: une contradiction est le cas contraire d'une formule satisfaisable

- **Tautologie:** une formule φ vraie pour toute valuation est dite valide ou bine une tautologie et on la note : $\models \varphi$
- **Invalide:** une formule φ est invalide si elle admet au moins une valuation pour laquelle φ est fausse

Note: une tautologie est le cas contraire d'une formule invalide

Sémantique du CP0: Satisfaisabilitié, Tautologie et Contradiction

- **Exemples**
  - La formule $((p \Rightarrow q) \wedge p) \Rightarrow p$ est une tautologie
  - La formule $(p \Rightarrow q) \wedge (p \wedge \neg q)$ est une contradiction
  - La formule $p \wedge (q \Rightarrow p)$ est satisfiable et invalide

Dressez la table de vérité pour chaque des formules et prouvez la conclusion

Remarque:

Pour n'importe quelle formule propositionnelle, il est possible de savoir si la formule est valide ($\models$), invalide, satisfaisable (SAT) ou insatisfaisable (INSAT)

► Il suffit de dresser la table de vérité

Sémantique du CP0: Satisfaisabilitié, Tautologie et Contradiction

Remarque:

- Si la formule est valide (tautologie) alors elle est satisfaisable. L'inverse n'est pas vrai.
- Si la formule est insatisfaisable (contradiction) alors elle est invalide. L'inverse n'est pas vrai.
- Une formule peut être à la fois satisfaisable (existe au moins une valuation pour laquelle la formule est vraie) et invalide (existe au moins une valuation pour laquelle la formule est fausse)
- Une formule ne peut jamais être à la fois valide (tautologie: formule toujours vraie) et insatisfaisable (formule toujours fausse)

Sémantique du CP0: Satisfaisabilitié, Tautologie et Contradiction

Proprietés:

- A est une tautologie ssi ⌉ A est une contradiction

Preuve : (1 er sens) ⌉ A est une contradiction si A une tautologie

A est une tautologie alors [A] = V pour toute valuation de A donc

[⌉ A] = ⌉ V = F pour toute valuation de ▫ ⌉ A est une contradiction

Preuve : (2 ème sens) A est une tautologie si ⌉ A est une contradiction

⌉ A est une contradiction cad [⌉ A] = F pour toute valuation d'où [A] = V pour toute valuation ▫ A est une tautologie

- A est une contradiction ssi ⌉ A est une tautologie

Preuve (même logique)

Sémantique du CP0: Satisfaisabilitié, Tautologie et Contradiction

Propriétés:

- (p ∨ ⌉ p) est une tautologie
- (p ∨ q1 ∨ … ∨ qn ∨ ⌉ p ∨ qn+1 ∨ … ∨ qn+m) est une tautologie
- (p ∨ A ∨ ⌉ p ∨ B) est une tautologie
- (p ∧ ⌉ p) est une contradiction
- (p ∧ q1 ∧ … ∧ qn ∧ ⌉ p ∧ qn+1 ∧ … ∧ qn+m) est une contradiction
- (p ∧ B ∧ ⌉ p ∧ C) est une contradiction

Proposition:

- Soient
- A , B1, B2, …, Bn des formules propositionnelles
- p1, p2, …, pn des variables propositionnelles
- Si A est une tautologie alors A [p1 ← B1 , p2 ← B2 , … ,pn ← Bn] est également une tautologie

Sémantique du CP0: Equivalence et Conséquence Logique

Définitions:

- Conséquence logique: une formule Ψ est une conséquence logique d'une formule φ ssi mod(φ) ⊆ mod (Ψ)
  - Notation: $\varphi \models \Psi$
- Equivalence : une formule φ est équivalente à une formule Ψ si:

  $\varphi \models \Psi$ et $\Psi \models \varphi$ c.à.d mod(φ) ⊆ mod (Ψ) et mod(Ψ) ⊆ mod (φ)
  - Notation $\varphi \equiv \Psi$

Sémantique du CP0: Equivalence et Conséquence Logique

Proprietés:

- 1. B ⊨ A ssi B ⇒ A est une tautologie (⊨ (B ⇒ A))
- 2. B ≡ A ssi B ⇔ A est une tautologie (⊨ (B ⇔ A))
- 3. Si B ≡ A et ⊨ B alors ⊨ A

Sémantique du CP0: Equivalence et Conséquence Logique

Preuve:

- 1. B ⊨ A ssi B ⇒ A est une tautologie (⊨ (B ⇒ A))

1 er sens: B ⊨ A si B ⇒ A est une tautologie

B ⇒ A est une tautologie ▫ mod (B ⇒A) ={(0,0),(1,1),(0,1)}

mod(B) ={1} ; mod(A)={1} alors mod (B) ⊆ mod (A) ▫ B ⊨ A (par définition)

2 -ème sens: B ⇒ A est une tautologie (⊨ (B ⇒ A) si B ⊨ A

B ⊨ A : A est une conséquence logique de B eq à mod (B) ⊆ mod (A), càd tout modèle de B existe dans A ainsi B est vraie alors A est vraie d'où B ⇒ A est vraie (tautologie)

Sémantique du CP0: Equivalence et Conséquence Logique

Preuve:

- 2. $B \equiv A$ ssi $B \Leftrightarrow A$ est une tautologie ($\models (B \Leftrightarrow A)$)

1 er sens: $B \equiv A$ si $B \Leftrightarrow A$ est une tautologie

$B \Leftrightarrow A$ est une tautologie donc $mod(B \Leftrightarrow A) = \{(0,0), (1,1)\}$ ▫ $mod(B)=mod(A)$ ▫ $B \equiv A$

2 -ème sens: $B \Leftrightarrow A$ est une tautologie si $B \equiv A$

$B \equiv A$ eq $mod(B)=mod(A)$ ▫

- $mod(B) \subseteq mod(A)$ ▫ $B \Rightarrow A$ (B vraie alors A est vraie)

et

- $mod(A) \subseteq mod(B)$ ▫ $A \Rightarrow B$ (A vraie alors B est vraie)

Donc $B \Leftrightarrow A$ est une tautologie

Sémantique du CP0: Equivalence et Conséquence Logique

Preuve:

- Si B ≡ A et ⊨ B alors ⊨ A

B ≡ A est vraie alors B ⇔ A est une tautologie {(0,0),(1,1)} et B est une tautologie (⊨ B) cad B est toujours vraie ▫ A est vraie (A est une tautologie)

Propriétés

1. Si A ≡ B alors ⌉ A ≡ ⌉ B
2. Si A ≡ B et C ≡ D alors
   - (A ∨ C) ≡ (B ∨ D)
   - (A ∧ C) ≡ (B ∧ D)
   - (A ⇒ C) ≡ (B ⇒ D)
   - (A ⇔ C) ≡ (B ⇔ D) (Traçer la table de vérité des propriétés)

Sémantique du CP0: Equivalence et Conséquence Logique

Equivalence matérielle vs Equivalence logique Définitions:

- **Equivalence logique:** est une notion d'équivalence qui concerne deux formules qui dans toutes les situations se comportent de la même façon. Il s'agit d'une relation entre formules, que l'on présente en moyen de symbole ≡

- **Equivalence matérielle:** est un opérateur étant donné deux opérandes (A ⇔ C), ne dit rien des formules A et C mais vaut vrai ou faux selon la valeur de ses opérandes.

Exemple:

Comparons les valeurs de la vérité de ces 2 formules:

A: (¬(¬p ∧ ¬q) ; B: p ∨ q

- Après le dressage de la table de vérité on a A et B identiques pour toutes valuation de p et q
- On dit que A et B sont logiquement équivalentes (A ≡ B)

Il est facile de vérifier que si A ≡ B alors A ⇔ B est toujours vraie. (logique matérielle)

Sémantique du CP0: Equivalence et Conséquence Logique

Théorème: Le calcul propositionnel est muni d'une structure d'algèbre de Boole ainsi que loi de Morgan.

- **Associativité** $A \vee (B \vee C) \equiv (A \vee B) \vee C$

  $A \wedge (B \wedge C) \equiv (A \wedge B) \wedge C$
- **Commutativité** $(A \vee B) \equiv (B \vee A)$

  $(A \wedge B) \equiv (B \wedge A)$
- **Distributivité** $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$

  $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
- **Lois de De Morgan** $\neg (A \vee B) \equiv \neg A \wedge \neg B$

  $\neg (A \wedge B) \equiv \neg A \vee \neg B$
- **Idempotence** $(A \vee A) \equiv A$

  $(A \wedge A) \equiv A$
- **Absorption** $A \wedge (A \vee B) \equiv A$

  $A \vee (A \wedge B) \equiv A$

Sémantique du CP0: Equivalence et Conséquence Logique

- **Eléments neutres**

  $(A \wedge \mathbf{V}) \equiv A \quad (A \vee \mathbf{F}) \equiv A$

- **Eléments absorbants**

  $(A \wedge \mathbf{F}) \equiv F \quad (A \vee \mathbf{V}) \equiv V$

- **Tiers exclu**

  $(A \wedge \neg A) \equiv F \quad (A \vee \neg A) \equiv V$

- **Inverse**

  $\neg V \equiv F \quad \neg F \equiv V$

- **Involution** $\neg \neg A \equiv A$

Sémantique du CP0: Equivalence et Conséquence Logique

- **Remarques:**

Un ensemble de formules peut être vu comme une conjonction de formules

- $\mathcal{E}$ = { A1, …, An } est sémantiquement équivalent à la formule (A1 ∧ … ∧ An)

Plus précisemment: { A1, …, An } ≡ { (A1 ∧ … ∧ An) }

- **Application:**

Soit l'énoncé suivant:

« **Si** *je travaille bien* **alors** *je vais réussir*. **Si** *je suis malade* , *je ne peux pas bien travailler*. *Je suis malade* **mais** *je travaille bien*. Donc *je vais réussir*.»

- 1. Définition des variables propositionnelles

t : « je travaille bien»

m : « je suis malade»

r : « je vais réussir»

Sémantique du CP0: Equivalence et Conséquence Logique

- 2. Modélisation de l'énoncé :

H1 : «**Si** *je travaille bien* **alors** *je vais réussir*» **t** ⇒ **r**

H2 : «**Si** *je suis malade* **,** *je ne peux pas bien travailler*» **m** ⇒ ⌉ **t**

H3 : «*je suis malade* **mais** *je travaille bien*» **m** ∧ **t**

C : «*je vais réussir*» **r**

- 3. Vérifier que l'énoncé est vrai: {H1, H2, H3} ⊨ C

{ t ⇒ r , m ⇒ ⌉ t , m ∧ t } ⊨ r

ssi (t ⇒ r) ∧ (m ⇒ ⌉ t) ∧ (m ∧ t) ⊨ r

ssi ⊨ (((t ⇒ r) ∧ (m ⇒ ⌉ t) ∧ (m ∧ t)) ⇒ r)

A verifier avec table de vérité (page suivante)

Sémantique du CP0: Equivalence et Conséquence Logique

m	t	r	H1: t ⇒ r	H2: m ⇒ ¬ t	H3: m ∧ t	1 ∧ 2 ∧ 3	1 ∧ 2 ∧ 3 ⇒ r
0	0	0	1	1	0	0	1
0	0	1	1	1	0	0	1
0	1	0	0	1	0	0	1
0	1	1	1	1	0	0	1
1	0	0	1	1	0	0	1
1	0	1	1	1	0	0	1
1	1	0	0	0	1	0	1
1	1	1	1	0	1	0	1

- H4 est toujours vraie quelque soit l'état de m, t et r du coup H4 est une tautologie

{ H1, H2, H3} ⊨ H4 d'où énoncé est vrai

Sémantique du CP0: Equivalence et Conséquence Logique

Les connecteurs logiques:

- **Conjonction:** $A \wedge B$
- **Disjonction :** $A \vee B$
- **Ou exclusif:** $A \oplus B$ $(A \wedge \neg B) \vee (\neg A \wedge B)$ / (A ou B mais pas les deux)
- **Implication:** $A \Rightarrow B$
- **Equivalence:** $A \Leftrightarrow B$

Sémantique du CP0: Formes Normales

Définition:

- **Littéral:** si p est un symbôle propositionnel et que p est un atome ⌉p sa négation alors tout ceci est un littéral
- FND formule normale disjonctive si elle est sous forme de disjonction de conjonctions des littéraux.

$$(l11 \wedge ... \wedge l1i) \vee (l21 \wedge ... \wedge l2j) \vee \ldots \vee (ln1 \wedge ... \wedge lnk)$$

Note: La conjonction de littéraux est appelée clause conjonctive

- FNC formule normale conjonctive si elle est sous forme de conjonction de disjonction des littéraux.

$$(l11 \vee ... \vee l1i) \wedge (l21 \vee ... \vee l2j) \wedge ... \wedge (ln1 \vee ... \vee lnk)$$

Note: La disjonction de littéraux est appelée clause disjonctive

Exemples

- Formule en FND : $(p \wedge \neg q) \vee p$
- Formule en FNC : $(p \vee q) \wedge (r \vee \neg s)$
- Formules à la fois en FNC et FND :

p $p \vee q$ $p \wedge \neg q$

Sémantique du CP0: Formes Normales

Algorithme de normalisation:

Les FNC et FND peuvent être obtenues en appliquant les transformations suivantes:

- 1. Elimination des connecteurs ⇔ et ⇒
- (A ⇔ B) ≡ (A ⇒ B) ∧ (B ⇒ A)
- (A ⇒ B) ≡ (¬ A ∨ B)
- 2. Elimination des négations appliquées à des sous-formules
- ¬ (¬ A) ≡ A
- ¬ (A ∧ B) ≡ (¬ A ∨ ¬ B)
- ¬ (A ∨ B) ≡ (¬ A ∧ ¬ B)
- Application des lois de distributivité tant que possible
- A ∨ (B ∧ C) ≡ (A ∨ B) ∧ (A ∨ C) (pour obtenir une FNC)
- A ∧ (B ∨ C) ≡ (A ∧ B) ∨ (A ∧ C) (pour obtenir une FND)

- Ceci nécessite l'application des lois d'associativité et decommutativité

Sémantique du CP0: Formes Normales

Théorème:

Toute formule admet une FNC et une FND qui sont sémantiquement équivalentes.

Remarque:

Les FNC et les FND pour une formule donnée ne sont pas uniques mais elles sont toutes sémantiquement équivalentes.

Exemple: Soit la formule ci-dessous:

- $\varphi = (\neg(p \wedge (\neg q \vee (r \vee s)))) \wedge (p \vee q).$

1- Mettre sous forme FND tout en suivant les étapes
2- Evaluez la formule
3- Déduire le type de la formule tout en justifiant